NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE

NTU Academy for Professional and Continuing Education

(SCTP) Advanced Professional Certificate

Data Science and AI

NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE

3.5 Unsupervised Learning

Module Overview

3.1 Probability and Statistics for Machine Learning

3.2 Introduction to Machine Learning

3.3 Supervised Learning

3.4 Supervised Learning - Advanced

3.5 Unsupervised Learning

3.6 Time Series Data & Forecasting

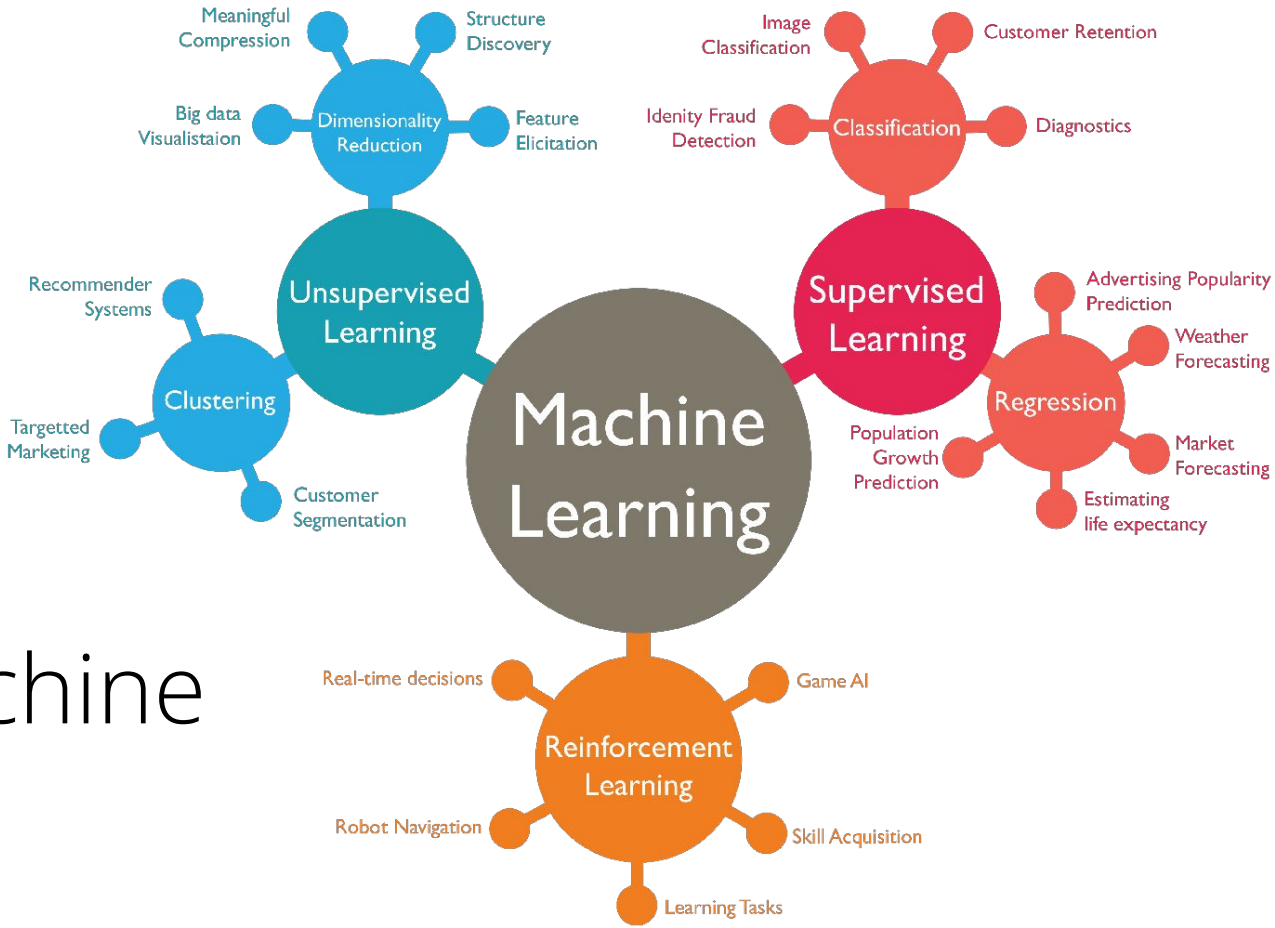
3.7 Neural Network & Deep Learning

3.8 Computer Vision (CV)

3.9 Natural Language Processing (NLP)

3.10 NLP - Advanced

Types of Machine Learning



Lesson Objectives

- Detect outliers and anomalies
- Perform dimensionality reduction to reduce the number of features
- Perform clustering to group similar data points together

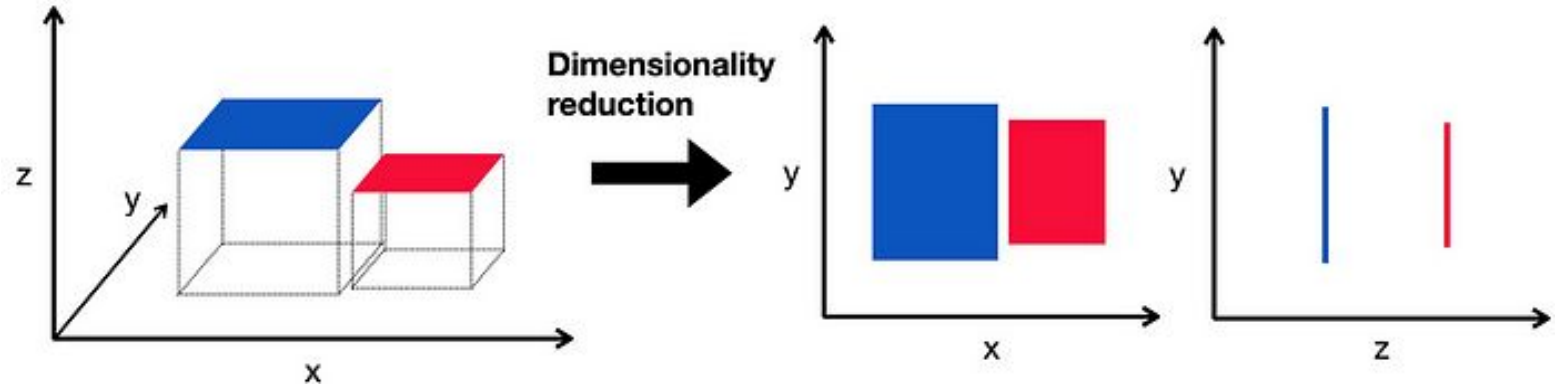
Lesson Plan

- **Dimensionality Reduction**
- Clustering

Dimensionality Reduction

Dimensionality reduction is a critical step in data preprocessing, especially for high-dimensional datasets. It helps in reducing the number of features, combating the curse of dimensionality, and improving model performance.

Two popular techniques for dimensionality reduction are Principal Component Analysis (**PCA**) and t-Distributed Stochastic Neighbor Embedding (**t-SNE**).



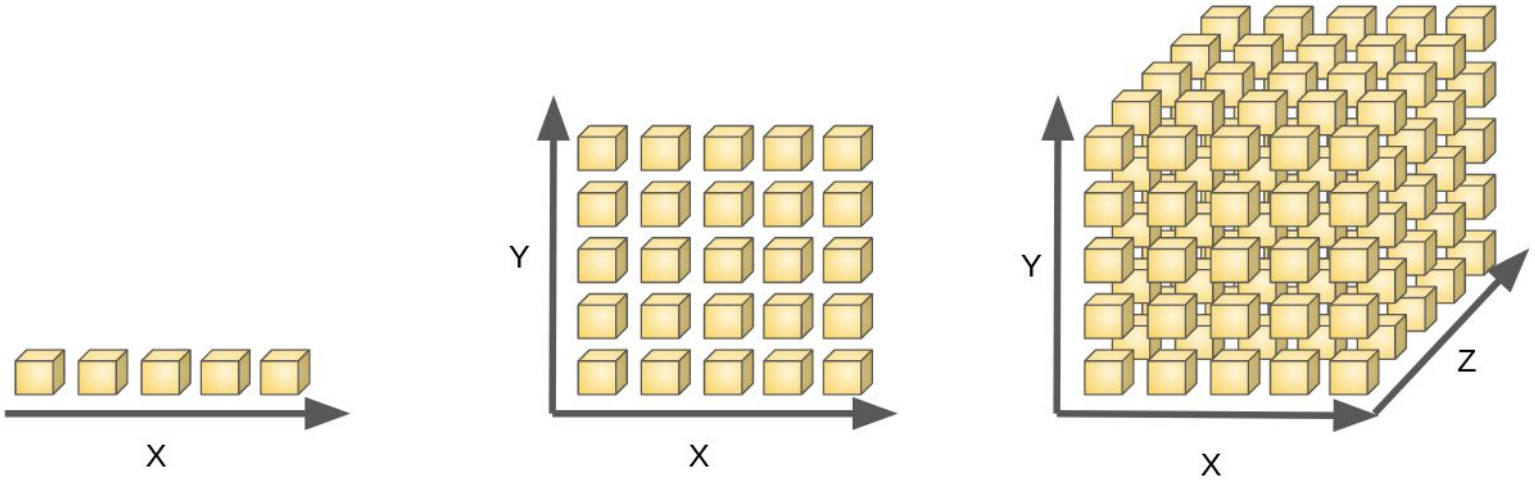
Why do we need to reduce dimensionality?

High-dimensional data can lead to several issues:

- **Curse of Dimensionality**: As the number of features increases, the amount of data needed to generalize accurately increases exponentially.
- **Overfitting**: Having too many features may cause the model to fit noise rather than the actual signal.
- **Data Visualization**: Visualizing data in high dimensions is impractical, making it difficult to gain insights.

By reducing the dimensionality, we can simplify models, reduce computation, and improve visualization without losing critical information.

Why do we need to reduce dimensionality?



Principal Component Analysis (PCA)

PCA is a linear dimensionality reduction technique that transforms the data into a new coordinate system such that the greatest variance by any projection of the data comes to lie on the first coordinate (called the first principal component), the second greatest variance on the second coordinate, and so on. It is **unitless**.

- **Data Compression**: PCA helps reduce the number of features while retaining the majority of the dataset's variance.
- **Noise Reduction**: It helps eliminate noise by focusing on the most significant components.
- **Visualization**: PCA is commonly used for visualizing high-dimensional data in 2D or 3D.

PCA Benefits

Pros

1. **Data Compression**
   Reduces features while preserving dataset variance effectively.
2. **Noise Reduction**
   Eliminates noise by focusing on significant components.
3. **Visualization**
   Simplifies high-dimensional data for 2D/3D visualization.

Cons

1. **Information Loss**
   May lose some information during feature reduction.
2. **Computational Cost**
   Can be computationally intensive for large datasets.

How to do PCA

Standardise the data:

This ensures that each feature contributes equally to the analysis.

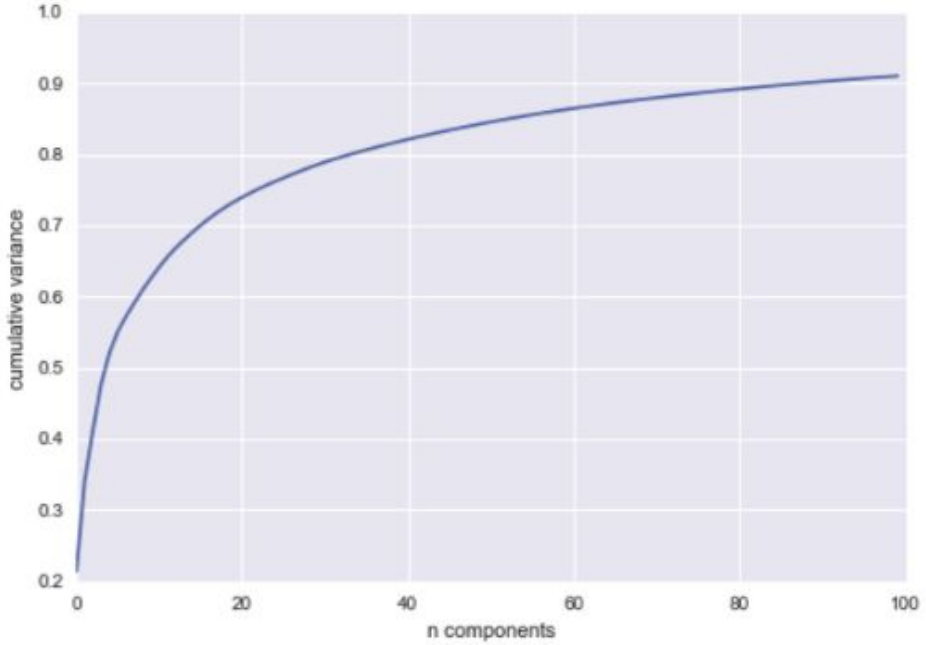
Considerations

- If your data contains outliers, **Min-Max** scaling can be sensitive to those extreme values, which might distort the distribution of data and affect the PCA results.
- **Standard Scaler** is generally more robust when the data contains outliers, as it scales based on the mean and standard deviation, which aren't heavily influenced by extreme values.
- If your data doesn’t have significant outliers and you're more interested in preserving the overall distribution of the data (with specific boundaries), Min-Max scaling can work well with PCA.
- But for data with outliers or if you want to focus on the variance in each feature (regardless of the scale), Standard Scaler might be the better choice.

Choose your *n*:

Trade-off between dimensionality reduction and the amount of variance retained - The fewer feature, lower computational complexity, less noise, easier to visualise, but may lose some pattern or relationship between the data

How to do PCA



$$\text{Explained Variance Ratio}_i = \frac{\lambda_i}{\sum_{j=1}^{n} \lambda_j}$$

Manifold Learning

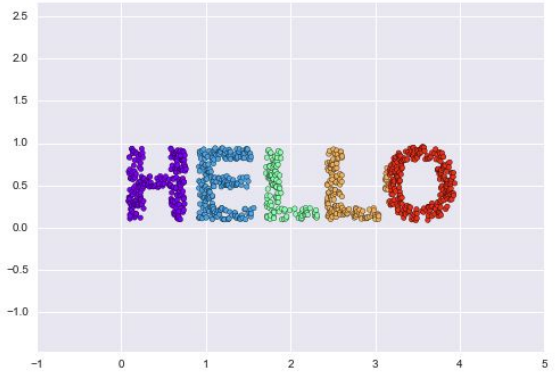
Why is manifold learning relevant?

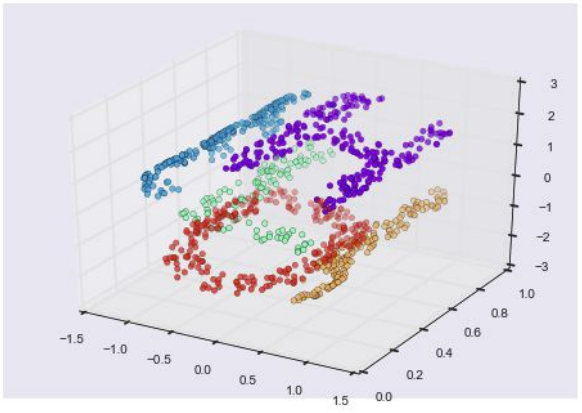
https://jakevdp.github.io/PythonDataScienceHandbook/05.10-manifold-learning.html

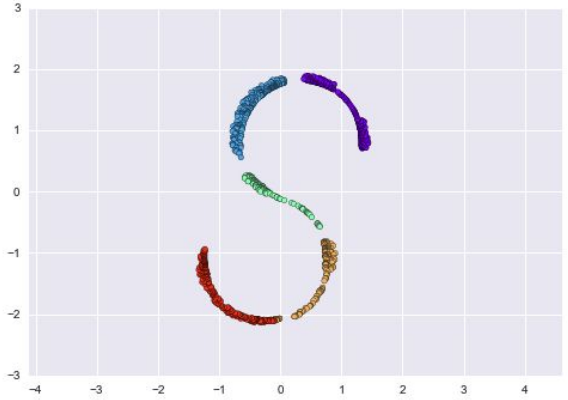
" When you think of a manifold, I'd suggest imagining a sheet of paper: this is a two-dimensional object that lives in our familiar three-dimensional world, and can be bent or rolled in that two dimensions. In the parlance of manifold learning, we can think of this sheet as a two-dimensional manifold embedded in three-dimensional space.

Rotating, re-orienting, or stretching the piece of paper in three-dimensional space doesn't change the flat geometry of the paper: such operations are akin to linear embeddings. If you bend, curl, or crumple the paper, it is still a two-dimensional manifold, but the embedding into the three-dimensional space is no longer linear. Manifold learning algorithms would seek to learn about the fundamental two-dimensional nature of the paper, even as it is contorted to fill the three-dimensional space. "

Manifold Learning - when linear projection fails







t-Distributed Stochastic Neighbor Embedding (t-SNE)

t-SNE is a non-linear dimensionality reduction technique that is particularly well suited for the visualization of high-dimensional datasets. It converts similarities between data points to joint probabilities and tries to minimize the Kullback-Leibler divergence between the joint probabilities of the low-dimensional embedding and the high-dimensional data.

- **Visualization**: t-SNE is widely used for visualizing high-dimensional data in 2D or 3D plots.
- **Capturing Non-Linear Relationships**: It excels at capturing the local structure and relationships within the data.
- **However,** t-SNE does not preserve distances and might give misleading insights if you are not careful

t-Distributed Stochastic Neighbor Embedding (t-SNE)

- More resources: https://distill.pub/2016/misread-tsne/

Code-along

Jupyter Notebook

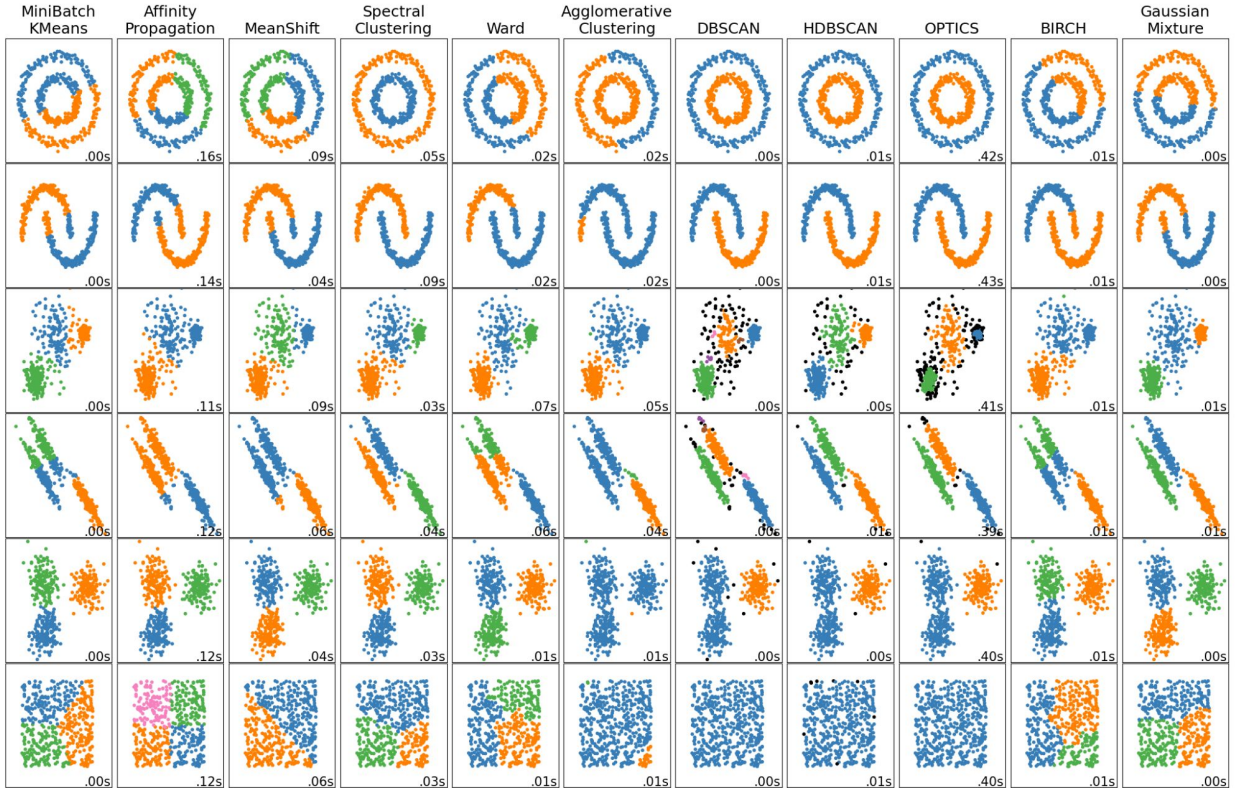
Part 2: Dimensionality Reduction

Lesson Plan

- Dimensionality Reduction
- **Clustering**

Clustering

Clustering is an unsupervised learning technique used to group similar data points together based on their features. If the examples are labeled, this kind of grouping is called classification.



K-Means Clustering

K-Means is a centroid-based clustering algorithm that partitions the data into K distinct clusters based on distance to the centroid of the clusters.

Inertia (within-cluster sum-of-squares criterion) is not a normalized metric: we just know that lower values are better and zero is optimal.

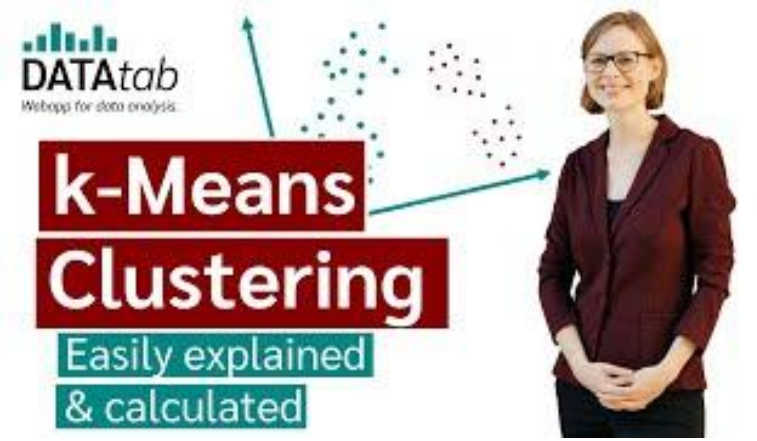
But in very high-dimensional spaces, Euclidean distances tend to become inflated (this is an instance of the so-called *"curse of dimensionality"*).

Running a dimensionality reduction algorithm such as Principal component analysis (**PCA**) prior to k-means clustering can alleviate this problem and speed up the computations.

How K-Means Works

- Initialize K centroids randomly.
- Assign each data point to the closest centroid, forming K clusters.
- Recalculate the centroids as the mean of all points in the cluster.
- Repeat steps 2 and 3 until convergence (i.e., assignments no longer change).

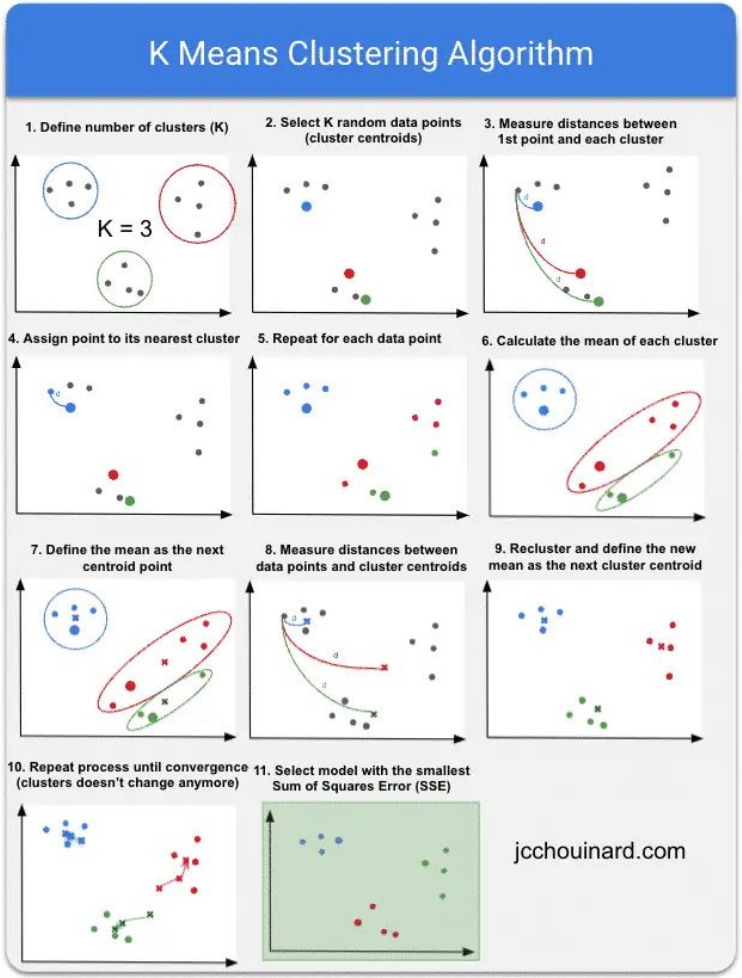
K-Means Clustering



https://youtu.be/GZj6ikx8PAc?si=bps8v3Ht6PH9Xjkw&t=19
(0:19 to 3:51)

How K-Means Works

- Initialize K centroids randomly.
- Assign each data point to the closest centroid, forming K clusters.
- Recalculate the centroids as the mean of all points in the cluster.
- Repeat steps 2 and 3 until convergence (i.e., assignments no longer change).



K-means clustering

Advantages	Disadvantages
✔ **Efficiency:** K-Means is computationally efficient, especially for a large number of data points, making it suitable for big datasets. ✔ **Simplicity:** The algorithm is straightforward to understand and implement. ✔ **Scalability:** It can easily adapt to new examples and is scalable with the use of algorithms like the mini-batch K-Means. ✔ **Well-suited for Spherical Clusters:** K-Means performs well when the clusters are spherical and well-separated.	✖ **Number of Clusters:** The number of clusters, K, must be specified beforehand, which can be difficult to estimate without prior knowledge of the data. ✖ **Sensitivity to Initial Centroids:** The final clusters can vary based on the initial choice of centroids. The algorithm can converge to local optima. ✖ **Sensitivity to Outliers:** Outliers can skew the clusters since K-Means tries to minimize the variance within each cluster. ✖ **Assumption of Spherical Clusters:** K-Means assumes that clusters are spherical and equally sized, which might not always be the case. ✖ **Difficulty with Different Densities:** It may not perform well when clusters have varying densities.

Hierarchical Clustering

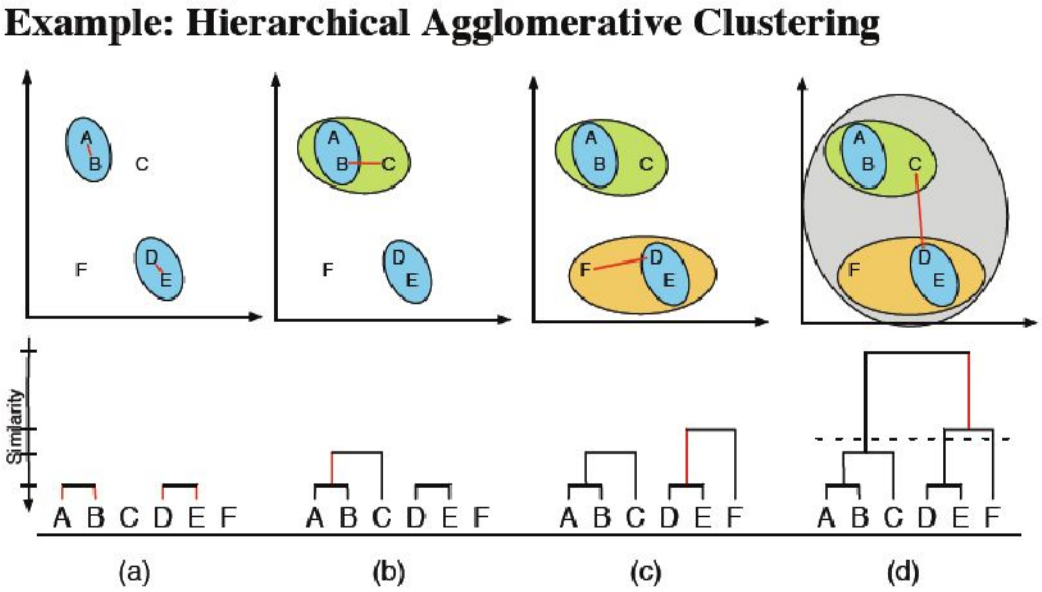
https://www.youtube.com/watch?v=8QCBl-xdeZI&t=235s

Hierarchical clustering

Hierarchical clustering is a method of cluster analysis that seeks to build a hierarchy of clusters.

The process can be approached in two ways: Agglomerative (bottom-up) and Divisive (top-down). Here, we'll focus on the more commonly used agglomerative approach.

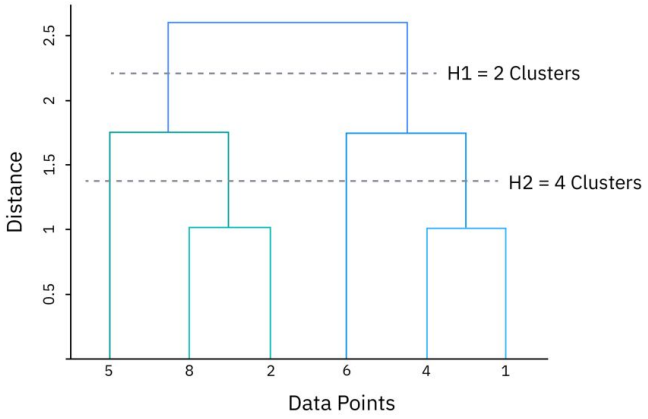
The root of the tree is the unique cluster that gathers all the samples, the leaves being the clusters with only one sample.

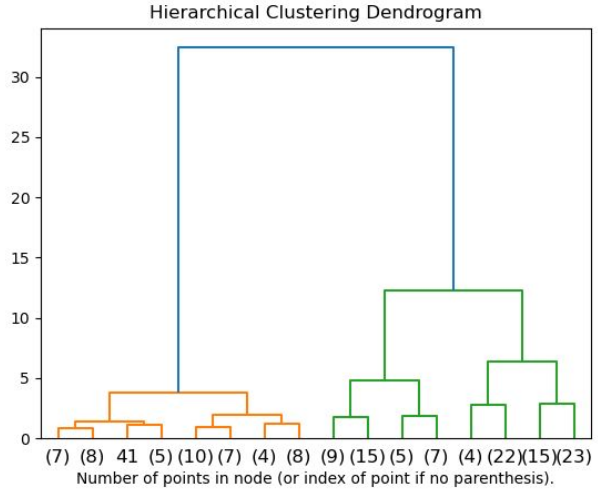


Hierarchical clustering

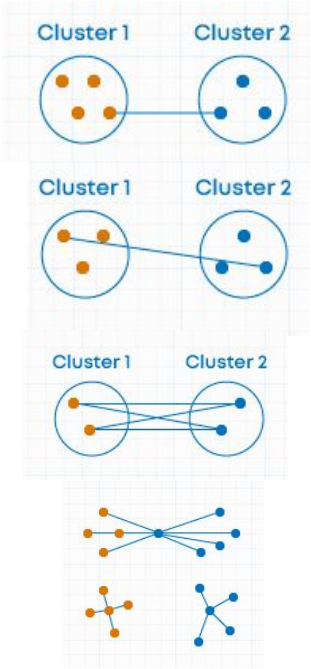
Hierarchical clustering is a method of cluster analysis that seeks to build a hierarchy of clusters. The process can be approached in two ways: Agglomerative (bottom-up) and Divisive (top-down). Here, we'll focus on the more commonly used agglomerative approach.

The root of the tree is the unique cluster that gathers all the samples, the leaves being the clusters with only one sample.





Linkage Type



Linkage Type	Cluster Shape	Sensitive to Outliers	Good For
Single Linkage	Arbitrary	Yes	Non-spherical clusters, chaining
Complete Linkage	Compact / spherical	Yes	Tight, small clusters
Average Linkage	Balanced	Somewhat	General-purpose
Ward's Method	Type (dense in middle)	Less	Equal-size clusters, low variance

Hierarchical Clustering

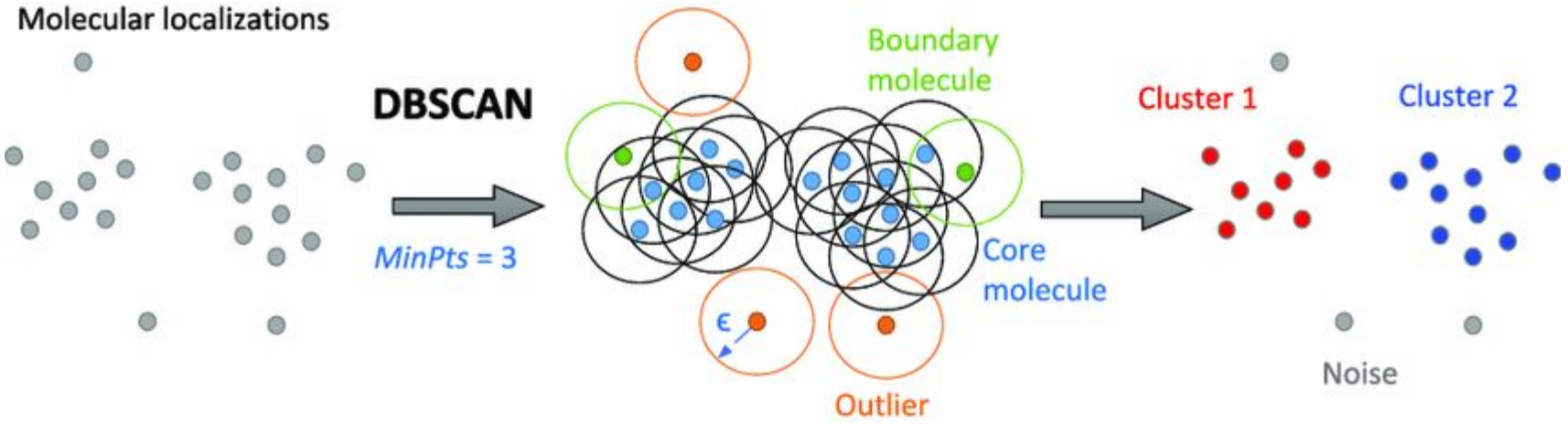
Advantages	Disadvantages
✔ **No Need to Specify Number of Clusters:** You do not need to specify the number of clusters beforehand. The number of clusters can be determined by cutting the dendrogram at the desired level.	✖ **Computational Complexity:** The time complexity for the standard algorithm is typically $O(n^2 \log n)$ or $O(n^3)$, which can be prohibitive for large datasets.
✔ **Dendrogram:** Provides a visual representation of the clustering process, which can be informative.	✖ **Irreversible:** Once a decision is made to combine two clusters, it cannot be undone without restarting the entire process.
✔ **Flexibility:** Can create clusters of various shapes and sizes, not limited to spherical clusters like K-Means.	✖ **Sensitivity to Noise and Outliers:** Similar to K-Means, hierarchical clustering can be sensitive to noise and outliers, which can lead to misinterpretations of the data structure.
✔ **Easy to Merge/Split Clusters:** The hierarchical nature allows for easy merging or splitting of clusters based on the hierarchy.	✖ **Choice of Linkage Criteria:** The results can be significantly different based on the choice of linkage criteria (single, complete, average, etc.), and there is no objective way to choose the best method.

DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) groups together points that are closely packed (dense regions) and labels as outliers the points that lie alone in low-density regions.

- Unlike K-Means, you don't need to specify the number of clusters upfront, and it can find arbitrarily shaped clusters.
- Create clusters in non linear separable data
- Can be used for Anomaly detection as well (noise / outlier)
- Hyperparameters : min points and epsilon

DBSCAN



DBSCAN Applications

- Anomaly detection (e.g. fraud detection, network intrusion)
- Geospatial clustering (e.g. clustering locations, earthquakes)
- Customer segmentation
- Image segmentation (small-scale)

Feature / Method	K-Means Clustering	Hierarchical Clustering	DBSCAN
Type	Partitional	Hierarchical (Agglomerative/Divisive)	Density-based
Need to specify k upfront	Yes	No (but can choose # of clusters post-hoc)	No
Cluster shape assumption	Spherical, equal size	Flexible	Arbitrary shapes
Handles noise / outliers	❌ Poor	❌ Poor	✅ Good (labels outliers as noise)
Scalability	✅ Fast on large datasets	❌ Slow on large datasets	⚠️ Medium (depends on indexing & density)
Deterministic?	❌ No (depends on random init)	✅ Yes (deterministic merges/splits)	✅ Yes (deterministic with fixed parameters)
Distance Metric	Euclidean (default)	Customizable (e.g. Euclidean, cosine)	Customizable (e.g. Euclidean, Manhattan)
Number of clusters	Fixed (k)	Dendrogram cut gives flexibility	Determined by density (ε and minPts)
Best use cases	Well-separated, spherical clusters	Small to medium datasets; visual analysis	Noisy data, clusters with irregular shapes
Not good for	Non-spherical or overlapping clusters	Very large datasets	Varying density clusters
Output	Hard cluster assignment	Dendrogram + cluster assignment	Core points, border points, noise

Code-along

Jupyter Notebook

Part 3: Clustering

Lesson Plan

- Dimensionality Reduction
- Clustering